6. רשת פיזית, יחידות, גרעין הרוחב ותאים

6.1 מערכת הקואורדינטות והיחידות

src/layerforge_l1/connectivity_repair.py:117 IMPLEMENTED MATHEMATICS

$$s = \frac{\text{sheet_size_mm}}{\text{outer_pixels}} = \frac{225}{576 \cdot 4} = \frac{225}{2304} = 0.09765625 \text{ mm/px}$$

הרשת הפיזית היא מערך ריבועי בגודל 2304×2304. אינדקס (y, x) הוא שורה ועמודה, ו־y גדל כלפי מטה, כמו בתמו
$p = 32 \cdot 4 = 128$ פיקסלים מכל צד, כלומר $128s = 12.5$ מ"מ, ושטח היצירה הוא $2048s = 200$ מ"מ.

DERIVED התאמת היעד לרשת: היעד הוא 2048 × 2048 פיקסלים. src/layerforge_l1/reveal_planner.py:161 מחשב
$\text{scale} = 2048 \,/\, 2048 = 1$, ולכן כל פיקסל יעד הוא בדיוק פיקסל פיזי אחד. התכנון מתבצע ישירות ברזולוציה של 0.0977 מ"מ, בלי
מחדש.

```
/reveal_planner.py:161-168
ative: np.ndarray, grid: PocConfig, fill: int) -> np.ndarray:
est-upsample an artwork-sized array into the full physical grid."""

 grid.analysis_pixels // native.shape[0]
d.perimeter_pixels
p.full((grid.outer_pixels, grid.outer_pixels), fill, dtype=native.dtype)
p, p:-p] = np.repeat(np.repeat(native, scale, axis=0), scale, axis=1)
out
```

6.2 מדרישת 1.2 מ"מ לגרעין של 13 פיקסלים

src/layerforge_l1/connectivity_repair.py:114

```
/connectivity_repair.py:117-130
pixel = poc.sheet_size_mm / poc.outer_pixels
d_mm = (
imization.nominal_connection_width_mm
ptimization.vectorization_safety_mm
ptimization.cad_conversion_safety_mm

ed_pixels = enforced_mm / mm_per_pixel
d_pixels = math.ceil(requested_pixels)
V kernels are centred on one pixel.  A seven-pixel physical-width
rement therefore uses a 7x7 footprint (radius three), not a 9x9
rint (radius four).  If ceil() produces an even number, round the
rint up once to retain a centred, conservative odd kernel.
size = enforced_pixels if enforced_pixels % 2 else enforced_pixels + 1
radius = (kernel_size - 1) // 2
```

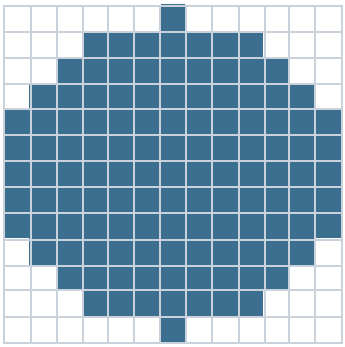
IMPLEMENTED MATHEMATICS

$$2 + 0.2 = 1.2 \text{ mm}, \quad n_{\text{req}} = \frac{w}{s} = \frac{1.2}{0.09765625} = 12.288, \quad n = \lceil n_{\text{req}} \rceil = 13, \quad k = \begin{cases} n & n \text{ odd} \\ n+1 & n \text{ even} \end{cases} = 13$$

הרוחב הפיזי של הגרעין: $13s = 1.2695$ מ"מ. העיגול כלפי מעלה תמיד שמרני, ולכן הדרישה בפועל מחמירה ב־0.07 מ"מ. גו
ב־OpenCV לגרעין יש פיקסל מרכזי (anchor), ורק עם גודל אי־זוגי השחיקה סימטרית לשני הצדדים.

src/layerforge_l1/reveal_planner.py:74 בונה את אותו גרעין למתכנן. כש־planning_margin_pixels=0, הגרעין של המת
ה־validator. זה מכוון: מה שהמתכנן מקבל, ה־validator יקבל.

הגרעין עצמו: `cv2.getStructuringElement(MORPH_ELLIPSE, (13,13))`



121 פיקסלים פעילים מתוך 169 (הופק מהקוד)

DERIVED **המשמעות הפיזית:** שחיקה ($\ominus$) בגרעין K משאירה פיקסל p רק אם כל הגרעין, כשמרכזו ב־p, נמצא בתוך החומר. לכן אנכי שורד את השחיקה רק אם רוחבו 13 פיקסלים לפחות (1.27 מ"מ). פס אלכסוני צריך להכיל את האליפסה, ו"רוחב" האליפסה בכיוו קטן מ־13 פיקסלים כי הפינות שלה חתוכות. לכן זו מדידה בדידה, לא מדידה רציפה מושלמת.

DERIVED **סף ה־nub:** `src/layerforge_l1/reveal_planner.py:253` : $\text{round}(0.5/0.09765625) = \text{round}(5.12) = 5$ צעד כלומר 0.49 מ"מ.

6.3 build_cells: חלוקת אזורים לתאים

`src/layerforge_l1/reveal_planner.py:85` . **מטרה:** לאפשר לאזור צבע אחד, למשל קו מתאר שחור שעובר בכל התמונה, להופיע ב אזור מוצב בעומק אחד, נוצרים מצבים שאין להם פתרון: בדוגמת הבדיקה (פרק 8.9) הווריאנט הזה נתקע.

```
/reveal_planner.py:95-127
 width = labels.shape
map = np.full(labels.shape, -1, dtype=np.int32)
color: list[int] = []
or in np.unique(labels):
ponents, count = ndimage.label(labels == color)
ion_map[components > 0] = components[components > 0] - 1 + len(region_color)
ion_color.extend([int(color)] * count)

config.split_regions:
urn (
 region_map,
 np.asarray(region_color, dtype=np.int32),
 np.arange(len(region_color), dtype=np.int32),


and fragment sizes are physical, so cells match at any resolution.
pixel = config.outer_sheet_mm * DEFAULT_CONFIG.analysis_pixels / DEFAULT_CONFIG.outer_pixels / width
max(2, int(round(config.cell_tile_mm / mm_per_pixel)))
els = max(1, int(round(config.min_cell_mm2 / mm_per_pixel**2)))
p = np.full(labels.shape, -1, dtype=np.int32)
gion: list[int] = []
n range(0, height, tile):
 x in range(0, width, tile):
 block = region_map[y : y + tile, x : x + tile]
 out = cell_map[y : y + tile, x : x + tile]
 for region in np.unique(block):
     components, count = ndimage.label(block == region)
     out[components > 0] = components[components > 0] - 1 + len(cell_region)
     cell_region.extend([int(region)] * count)

p, cell_region = _merge_small_cells(cell_map, cell_region, min_pixels)
gion_array = np.asarray(cell_region, dtype=np.int32)
cell_map, np.asarray(region_color, dtype=np.int32)[cell_region_array], cell_region_array
```

שלבי האלגוריתם

1. **אזורים (שורות 98–101):** לכל צבע, `ndimage.label` מוצא רכיבים 4־קשירים. כל רכיב מקבל מזהה אזור רץ, ו־`region_color` ש
2. **אריחים (שורות 110–123):** `src/layerforge_l1/reveal_planner.py:112` . התמונה נחתכת לרשת אריחים ריבועית. בכל אריח, 4־קשיר של החיתוך הופך לתא נפרד.

3. **מיזוג תאים זעירים (שורה 125):** `src/layerforge_l1/reveal_planner.py:130`. תא קטן מ־`min_pixels` מצורף לשכן **מאותו א**
גדול ביותר (טבעת 3×3, 8 שכנים). חוזרים על זה עד שאין שינוי, ואז מספרי התאים נדחסים מחדש.

IMPLEMENTED MATHEMATICS (שורות 111–113, ערכי Option B)

$$\frac{25 \cdot 512}{76 \cdot 2048} = 0.09765625, \quad \text{tile} = \max\left(2, \text{round}(2.0/s')\right) = \text{round}(20.48) = 20 \text{ px} = 1.95 \text{ mm}$$

$$\text{min_pixels} = \max\left(1, \text{round}(0.9/s'^2)\right) = \text{round}(94.37) = 94 \text{ px} \approx 0.896 \text{ mm}^2$$

הנוסחה ל־s' מבטאת את גודל שטח היצירה ($225 \cdot 512/576 = 200$ מ"מ) חלקי רוחב היעד, ולכן גודל התא **פיזי** ואינו תלוי ברזולוצי

תוצאה ב־35: OBSERVED **Option B** אזורים חולקו ל־11,498 תאים.

DERIVED **למה חותכים לפי אריחים ולא לפי מבנה הצורה?** החיתוך גאומטרי ועיוור לתוכן. הוא מבטיח שכל תא קטן מ־2 × 2 מ"מ,
"לפצל" כל אזור בגרנולריות הזאת. המחיר הוא "תפרים" (seams): גבול בין שני תאים של אותו אזור שהוצבו בעומקים שונים. מבחינה
כמעט לא נראה, כי שני הצדדים באותו צבע, אבל פיזית יש שם מדרגה של עובי לוח. האורך הכולל מדווח ב־`seam_report`.

LIMITATION כשאין לתא זעיר שכן מאותו אזור (למשל רכיב זעיר שכולו בתוך אריח אחד), הוא נשאר תא קטן. זה לא שגיאה, אבל המ
לתמוך בו כמו בכל תא אחר.